编程实验 2

1.题目：

班上 5 个学生（学号，姓名，数学成绩），有若干同学成绩不及格，采用双链表编程实现 下面的问题：

函数 1：动态建立双链表输入 5 个同学的数据。

函数 2：通过 next 指针输出双链表中不及格同学的数据（成绩<60）。

函数 3：通过 prior 指针输出双链表中及格同学的数据。

源程序：

实验-1.c 实验-2.c

计算机软件基础-实验-2 (全局范围)

```c
#define _CRT_SECURE_NO_WARNINGS
/*刘超然-2023060904025-题目1*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 5

typedef struct student {
    int ID;                 // 学号
    char name[50];          // 姓名
    float mathValue;        // 数学成绩
    struct student* prior;  // 指向下一个学生的指针
    struct student* next;   // 指向前一个学生的指针
}student;

student* doubleList();                  // 创建双向链表
void printFailStudents(student* head);  // 通过next指针输出不及格学生信息
void printPassStudents(student* tail);  // 通过prior指针输出及格学生信息

int main() {
    // 动态建立双链表，输入5个学生的数据
    student* head = doubleList();

    // 通过next指针输出双链表中不及格同学的数据
    printf("不及格的学生信息:\n");
    printFailStudents(head);

    student* tail = head;
    while (tail->next != NULL) {
        tail = tail->next;
    }

    // 通过prior指针输出双链表中及格同学的数据
    printf("\n及格的学生信息:\n");
    printPassStudents(tail);

    return 0;
}

// 函数1：动态建立双链表，输入5个同学的数据
student* doubleList() {
    student* head = NULL, * tail = NULL;
    for (int i = 0; i < MAX; i++) {
        student* newStudent = (student*)malloc(sizeof(student));
        printf("请输入第%d个学生的学号、姓名和数学成绩:", i + 1);
        scanf("%d %s %f", &newStudent->ID, newStudent->name, &newStudent->mathValue);

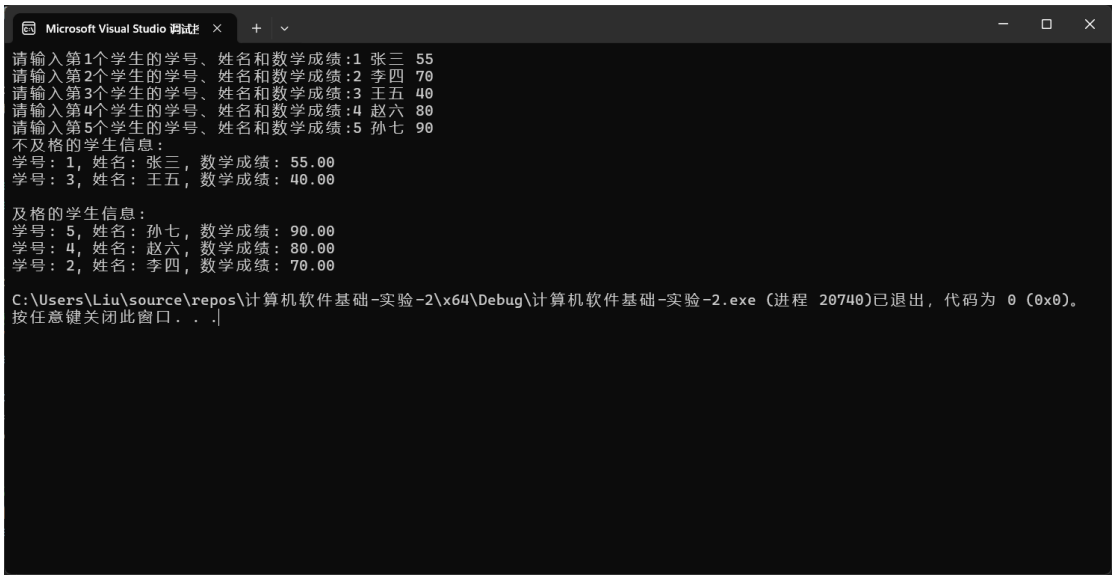
        newStudent->next = NULL;        // 初始化next指针为NULL
        newStudent->prior = tail;       // prior指向前一个学生

        if (tail != NULL) {
            tail->next = newStudent;    // 上一个学生的next指针指向新学生
        }
        else {
            head = newStudent;          // 第一个学生，作为头节点
        }
        tail = newStudent;              // 更新尾指针
    }
    return head;
}

// 函数2：通过next指针输出双链表中不及格同学的数据
void printFailStudents(student* head) {
    student* current = head;
    while (current != NULL) {
        if (current->mathValue < 60) {
            printf("学号: %d, 姓名: %s, 数学成绩: %.2f\n", current->ID, current->name, current->mathValue);
        }
        current = current->next; // 使用next指针遍历
    }
}

// 函数3：通过prior指针输出双链表中及格同学的数据
void printPassStudents(student* tail) {
    student* current = tail;
    while (current != NULL) {
        if (current->mathValue >= 60) {
            printf("学号: %d, 姓名: %s, 数学成绩: %.2f\n", current->ID, current->name, current->mathValue);
        }
        current = current->prior; // 使用prior指针遍历
    }
}
```

运行结果：



心得体会：

在这个实验中，通过使用双向链表管理学生信息，加深了我对链表及指针操作的理解，特别是在双向链表的动态创建、正反向遍历中的指针使用。在实现过程中，我通过 next 和 prior 指针分别输出不及格和及格学生的数据，对双向链表在数据遍历方面的灵活性有了更深刻的体会。同时，通过函数的模块化设计，代码的逻辑更为清晰，便于调试和扩展。遇到的问题也让我意识到双向链表中指针的正确指向和内存管理的重要性。在不断调试和修正的过程中，我培养了良好的编程规范和调试习惯。

2.题目：

某年级有两个班级，每个班级 5 个学生（学号，姓名，数学成绩）。采用循环链表编程实 现下面的问题：

函数 1：为每个班建立一个循环单链表，并且输入其成绩。

函数 2：实现两个循环单链表的连接。

函数 3：统计该年级的数学平均成绩。

源程序：

实验-2.c 实验-1.c

计算机软件基础-实验-2 (全局范围)

```c
#define _CRT_SECURE_NO_WARNINGS
/*刘超然-2023060904025-题目2*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 5

typedef struct student {
    int ID;                 // 学号
    char name[50];          // 姓名
    float mathValue;        // 数学成绩
    struct student* next;   // 指向下一个学生的指针
} student;

// 函数声明
student* classList(int classNumber);
student* classesConnect(student* class1, student* class2);
float averageCalculate(student* head);

int main() {
    // 创建两个班级
    student* class1 = classList(1);
    student* class2 = classList(2);

    // 连接两个班级的循环链表
    student* allStudents = classesConnect(class1, class2);

    // 统计数学平均成绩
    float average = averageCalculate(allStudents);
    printf("该年级数学平均成绩为: %.2f\n", average);

    // 释放内存
    student* current = allStudents;
    if (current) {
        student* temp;
        do {
            temp = current;
            current = current->next;
            free(temp);
        } while (current != allStudents);
    }

    return 0;
}

// 为每个班级建立一个循环单链表，并输入成绩
student* classList(int classNumber) {
    student* head = NULL;
    student* tail = NULL;
    for (int i = 0; i < MAX; i++) {
        student* newStudent = (student*)malloc(sizeof(student));
        printf("请输入班级%d 第%d个学生的学号、姓名和数学成绩: ", classNumber, i + 1);
        scanf("%d %s %f", &newStudent->ID, newStudent->name, &newStudent->mathValue);
        newStudent->next = newStudent;          // 指向自己，初始化为循环链表

        if (head == NULL) {
            head = newStudent;                  // 第一个学生
            tail = newStudent;
        }
        else {
            tail->next = newStudent;            // 将新学生链接到链表
            tail = newStudent;                  // 更新尾指针
        }
        tail->next = head;                      // 形成循环
    }

    return head;
}

// 实现两个循环单链表的连接
student* classesConnect(student* class1, student* class2) {
    if (class1 == NULL) return class2;          // 如果第一个班级为空，返回第二个班级
    if (class2 == NULL) return class1;          // 如果第二个班级为空，返回第一个班级

    student* class1Tail = class1;
    student* class2Tail = class2;

    // 找到第一个班级的尾节点
    while (class1Tail->next != class1) {
        class1Tail = class1Tail->next;
    }

    // 找到第二个班级的尾节点
    while (class2Tail->next != class2) {
        class2Tail = class2Tail->next;
    }

    // 连接两个班级
    class1Tail->next = class2;                  // 第一个班级的尾指针指向第二个班级的头
    class2Tail->next = class1;                  // 第二个班级的尾指针指向第一个班级的头

    return class1;                              // 返回连接后的第一个班级
}

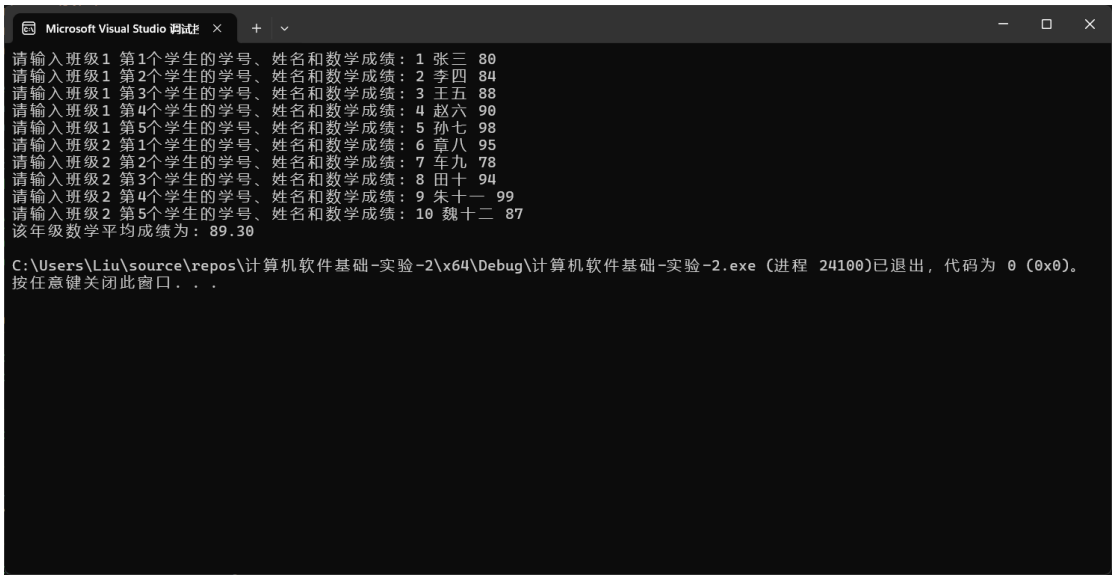
// 统计该年级的数学平均成绩
float averageCalculate(student* head) {
    if (head == NULL) return 0;

    float totalScore = 0;
    int count = 0;
    student* current = head;

    do {
        totalScore += current->mathValue;
        count++;
        current = current->next;
    } while (current != head);

    return totalScore / count;                  // 返回平均成绩
}
```

运行结果：



心得体会：

在这个实验中，通过使用 C 语言实现循环链表，管理学生的数学成绩数据，我不仅加深了对循环链表结构的理解，也掌握了链表节点的动态分配和指针操作的要点。尤其是通过创建和连接两个循环链表，并统计数学平均成绩，我对循环链表的特性和使用场景有了更直观的认识。实验过程中，通过结构体定义和函数封装提高了代码的可读性和模块化设计，也帮助我更熟练地使用指针操作来进行链表遍历、连接和数据统计。